



Exception & Error Handling

C++ Language

Exception & Error Handling

- ❖ When error codes are used to report errors
 - By default the caller of a function is free to use or ignore returned error codes
 - MISRA C/C++: returned values of function calls shall be used or explicitly discarded.
 - The `[[nodiscard]]` attribute can be used
 - `[[nodiscard]] int func(void) { return 10; }`
- ❖ There are 4 strategies that can be used:
 - Handle errors within the function
 - Pass errors back to the caller to deal with
 - Terminate the program
 - **Throw an exception**
- ❖ If in a function many things go wrong, error handling can be complex and even returned error codes may be ignored.

```
void printDivision(int x, int y) {  
    if (y != 0) { std::cout << (x / y); }  
    else { std::cout << "Error: Could not divide by zero"; }  
}  
  
bool doDivision(int x, int y, int &result) {  
    bool status = false; if (y != 0) { status = true; result = (x / y); }  
    return status;  
}  
  
int doDivision(int x, int y) {  
    if (y == 0) { std::cout << "Error: divide by zero"; std::exit(1); }  
    return (x / y);  
}
```

Exception & Error Handling

- ❖ Exception handling provides a mechanism to decouple handling of errors from the typical control flow of code. This allows more freedom to handle errors when and how ever is most useful for a given situation.
- ❖ In C++, exceptions are used to signal errors that cannot be handled locally, such as the failure to acquire a resource.
- ❖ Exceptions in C++ are implemented using three keywords: **throw**, **try**, and **catch**.
- ❖ In C++, a **throw statement** is used to signal that an exception/error has occurred.
- ❖ A **throw** statement, simply is the throw keyword, followed by a value of any object used to signal that an error has occurred

```
throw -1; // throw a literal integer value
throw ENUM_INVALID_INDEX; // throw an enum value
throw "Can not take square root of negative number"; // throw a literal C-style (const char*) string
throw dX; // throw a double variable that was previously defined
throw MyException("Fatal Error"); // Throw an object of class MyException
```

Exception & Error Handling

- ❖ The **try** keyword is used to define a block of statements. A **try block** acts as an observer to detect any exceptions that are thrown by any of the statements within the try block.
- ❖ And exception handling is done in **catch block(s)**. The **catch** keyword is used to define a block of code that handles exceptions for a single data type.
- ❖ Try blocks and catch blocks work together.
- ❖ A try block detects any exception that are thrown by statements within the try block, and routes it to an appropriate catch block for handling.

```
try {  
    // Statements that may throw exceptions you want to handle go here  
    throw -1; // here's a trivial throw statement  
}  
catch (int x) {  
    // Handle an exception of type int here  
    std::cout << "We caught an int exception with value" << x << std::endl;  
}
```

Exception & Error Handling

- ❖ A try block must have at least one catch block immediately following it, but may have multiple catch blocks listed in sequence.
- ❖ Once an exception has been caught by a try block and routed to a catch block, the exception is considered as handled.
- ❖ Catch parameters are like function params. Exceptions of fundamental types can be caught by value, but exceptions of other types should be caught by const reference/pointer in order to avoid making copies.

```
#include <cmath>
#include <iostream>
int main(void) {
    std::cout << "Enter a number: ";
    double x; std::cin >> x;
    try {
        // If the user entered a negative number, this is an error condition
        if (x < 0.0) {
            // throw exception of type const char*
            throw "Can not take sqrt of negative number";
        }
        std::cout << "The sqrt of " << x << " is " << std::sqrt(x) << std::endl;
    }
    catch (const char *exception) { // catch exceptions of type const char*
        std::cout << "Error: " << exception << std::endl;
    }
    return 0;
}
```

Exception & Error Handling

- ❖ In a catch block, if the parameter is not used, the parameter name can be omitted
- ❖ When an exception is thrown, execution of the program immediately jumps to the nearest enclosing try block
- ❖ If one of the catch handlers attached to a try block handles a type of exception, the handler is executed and the exception is considered as handled
- ❖ If no appropriate catch handler exists, the exception is propagated to the next enclosing try block
 - Propagating up the stack if necessary to find a try block
- ❖ If no appropriate catch handler can be found, the program will fail with an exception error and terminated.

```
#include <iostream>

int main(void) {
    try {
        try { throw -1; }
        catch (double) {
            std::cout << "double" << std::endl;
        }
    }
    catch (int) {
        std::cout << "int" << std::endl;
    }
    return 0;
}
```

Exception & Error Handling

- ❖ The compiler will not perform implicit conversions or promotions when matching exceptions with catch blocks. However, cast from derived classes to base classes will be performed.
- ❖ If an exception is routed to a catch block, it is considered as **handled**
 - Even if the catch block is empty.
- ❖ A catch block may throw an exception
 - The newly thrown exception is handled by the next enclosing try block; not the preceding try block
 - Because the catch block is outside of the try block
- ❖ It is possible to throw an exception out of a try block
 - Exceptions propagate up the stack to find a try block

```
#include <iostream>

int main(void) {
    try { throw -1; }
    catch (double) { std::cout << "double" << std::endl; }

    try { /* ... */ }
    catch (int) { std::cout << "int" << std::endl; }
    return 0;
}
```


Exception & Error Handling

- ❖ **Stack unwinding** is the process of removing function entries from stack at run time.

When an exception is thrown and control passes from a try block to a handler, destructors for all local objects constructed since the beginning of the try block are called. This process is called

stack unwinding. The auto objects are destroyed in reverse order.

```
#include <iostream>
void last() { // called by third()
    std::cout << "Start last" << std::endl;
    std::cout << "last throwing an int exception" << std::endl;
    throw -1; std::cout << "End last" << std::endl;
}
void third() { // called by second()
    std::cout << "Start third" << std::endl;
    last(); std::cout << "End third" << std::endl;
}
void second() { // called by first()
    std::cout << "Start second" << std::endl;
    try { third(); }
    catch (double)
    { std::cout << "second caught a double exception";
      std::cout << std::endl; }
    std::cout << "End second" << std::endl;
}
```

```
void first() { // called by main
    std::cout << "Start first" << std::endl;
    try { second(); }
    catch (int) {
        std::cout << "first caught an int exception" << std::endl;
    }
    catch (double) {
        std::cout << "first caught a double exception";
        std::cout << std::endl;
    }
    std::cout << "End first" << std::endl;
}
int main(void) {
    std::cout << "Start main" << std::endl;
    try { first(); }
    catch (int) { std::cout << "main caught an int exception";
                 std::cout << std::endl; }
    std::cout << "End main" << std::endl;
    return 0;
}
```


Exception & Error Handling

- ❖ Catch all types of exception using the ellipsis operator (...)
- ❖ A catch block of the form of catch(...) must be the last catch block
- ❖ It is possible to throw an exception in a catch block
 - To rethrow a caught exception use **throw;**

```
#include <iostream>

int main(void) {
    try {
        throw 5; // throw an int exception
    } catch (double x) {
        std::cout << "Caught an exception of type double: " << x << std::endl;
    } catch (...) { // catch-all handler
        std::cout << "Caught an exception of an undetermined type" << std::endl;
    }

    return 0;
}
```

```
#include <iostream>

int main(void) {
    try {
        try {
            throw 5; // Throw an int exception
        } catch (int x) {
            std::cout << "The caught exception: " << x << std::endl;
            throw "Throw a new exception in the catch block";
        }
    } catch (const char *e) {
        std::cout << e << std::endl;
        throw; // Rethrow the exception
    }
    catch (const char *e) { std::cout << e << std::endl; }

    return 0;
}
```

Exception & Error Handling

- ❖ Exceptions can be used in member functions of classes.
- ❖ When an exception is thrown in a constructor of a class the object's construction gets aborted and all the initialized class members get destructed automatically.

But the class destructor is never called and we cannot rely on the destructor to clean up resources.

```
#include <iostream>
class A { int x;
public: A(int _x) : x{_x} { std::cout << "A constructor" << std::endl;}
    ~A() { std::cout << "A destructor" << std::endl; }
};
class B { A a;
public: B(int x) : a{x} { std::cout << "B constructor" << std::endl; throw 10; }
    ~B() { std::cout << "B destructor" << std::endl;}
};
int main(void) { try { B b{10}; } catch (int e) { std::cout << e; } return 0; }
```

```
#include <iostream>
class MyArray {
    static constexpr int SIZE{5};
    int array[SIZE]{0};
public: MyArray()
{ static_assert(SIZE > 1, "SIZE shall be greater than 1"); }
    int &operator[](const int index) {
        if (index < 0 || index >= SIZE) {
            std::string tmp{std::to_string(index)};
            tmp += " is an invalid index!";
            throw tmp;
        }
        return array[index];
    }
};

int main(void) {
    try { MyArray arr; std::cout << arr[8] << std::endl; }
    catch (const std::string &e) { std::cout << e << std::endl; }
    return 0;
}
```

Exception & Error Handling

- ❖ A function-try-block associates a sequence of catch clauses with the entire function body, and with the member initializer list (if used in a constructor) as well.

```
#include <iostream>

class A {
    int x;
public: A(int _x) : x{_x} { if (x <= 0) { throw 1; } }
};

class B : public A {
public: B(int x) try : A{x} {}
    catch (int) { std::cout << "x shall be greater than 0" << std::endl; }
};

int main(void) {
    try { B b{0}; }
    catch (int) { std::cout << "Exception caught" << std::endl; }
    return 0;
}
```

```
#include <iostream>

class A {
    int x;
public:
    A(int _x) : x{_x} {
        if (x <= 0) { throw 1; }
    }
};

class B : public A {
public:
    B(int x) : A{x}
    { /* How can we catch the exception if creation of A fails? */ }
};

int main(void) {
    try { B b{0}; }
    catch (int) { std::cout << "Exception caught" << std::endl; }
    return 0;
}
```

Exception & Error Handling

- ❖ In a function try block, scope of the function parameters is extended to the end of the handlers.
- ❖ Function try blocks can catch both the base and the derived classes exceptions
- ❖ Unlike normal catch blocks, in a catch block of a function try block, we must throw/rethrow an exception. If we do not explicitly throw or rethrow an exception, the current exception will be implicitly rethrown.
- ❖ Don't use function try blocks to clean up resources. Function try blocks are useful primarily for either logging failures before passing the exception up the stack, or for changing type of the thrown exception.

```
#include <iostream>

class A {
    int x;
public: A(int _x) : x{_x} { if (x < 0) { throw 1; } }
};

class B : public A {
    int y;
public: B(int x, int _y) try : A{x}, y{_y}
    { if (y > 0) { throw false; } }
    catch (int) { std::cout << "x shall not be lesser than 0" << std::endl; }
    catch (bool) { std::cout << "y shall not be greater than 0" << std::endl; }
};

int main(void) {
    try { B b{-10, -10}; } catch (...) { std::cout << "x exception caught" << std::endl; }
    try { B b{10, 10}; } catch (...) { std::cout << "y exception caught" << std::endl; }
    return 0;
}
```

Exception & Error Handling

- ❖ Since it is possible to throw objects of any type and the compiler performs only implicit type casting from derived classes to base classes, handlers for derived classes exceptions shall be listed before handlers for base classes.

```
#include <iostream>

class Base {};

class Derived : public Base {};

int main()
{
    /* The exception is caught by handler for Derived */
    try { throw Derived{}; }
    catch (const Derived &derived) { std::cout << "caught Derived" << std::endl; }
    catch (const Base &base) { std::cout << "Caught Base" << std::endl; }
    return 0;
}
```

```
#include <iostream>

class Base {};

class Derived : public Base {};

int main()
{
    /* The exception is caught by handler for Base */
    try { throw Derived{}; }
    catch (const Base &base)
    { std::cout << "Caught Base" << std::endl; }
    catch (const Derived &derived) { std::cout << "caught Derived" << std::endl; }
    return 0;
}
```

Exception & Error Handling

- ❖ Possible to create exception classes and provide more info about errors
- ❖ There are some [standard exception classes](#) which are base on `std::exception`. For example a failed `dynamic_cast` throws `std::bad_cast` or the `new` operator may throw `std::bad_alloc`

```
#include <iostream>
#include <exception>

struct A { virtual ~A() = default; };
struct B { virtual ~B() = default; };

int main(void) {
    try { B b; A &a = dynamic_cast<A &>(b); }
    catch (const std::bad_cast &e) /* The type can also be std::exception */
    { std::cout << e.what() << std::endl; }
    return 0;
}
```

```
#include <iostream>

class MyException {
    int errcode; std::string errmsg;
public:
    MyException(int err_code, std::string err_msg) :
        errcode{err_code}, errmsg{err_msg} {}
    const char *getErrorMessage(void) const { return errmsg.c_str(); }
    int getErrorCode(void) const { return errcode; }
};

int main(void) {
    try { throw MyException{10, "My exception"}; }
    catch (const MyException &e)
    {
        std::cout << "Error Code: " << e.getErrorCode() << std::endl;
        std::cout << "Error Message: " << e.getErrorMessage() << std::endl;
    }
    return 0;
}
```

Exception & Error Handling

- ❖ It possible to use the standard exceptions directly or derive your own classes from `std::exception` or the standard exception classes.

```
#include <iostream>
#include <stdexcept>

class MyException : public std::exception {
    int errcode; std::string errmsg;
public:
    MyException(int err_code, std::string err_msg) : errcode{err_code}, errmsg{err_msg} {}
    const char *what(void) const noexcept override { return errmsg.c_str(); }
    int getErrorCode(void) const { return errcode; }
};

int main(void) {
    try { throw MyException{10, "My exception"}; }
    catch (const MyException &e) {
        std::cout << "Error Code: " << e.getErrorCode() << std::endl;
        std::cout << "Error Message: " << e.what() << std::endl;
    }
    return 0;
}
```

```
#include <iostream>
#include <stdexcept>

int main(void)
{
    try
    {
        throw std::runtime_error("Bad things happened");
    }
    // This handler catches std::exception and all the derived exceptions
    catch (const std::exception &exception)
    {
        std::cout << "Exception: " << exception.what() << std::endl;
    }

    return 0;
}
```


Exception & Error Handling

- ❖ Functions are classified as either non-throwing (do not throw exceptions) or potentially throwing (may throw exceptions).
- ❖ The `noexcept` specifier defines a function as non-throwing.
 - E.g. `void func(void) noexcept { /* the function body*/ }`
- ❖ Note that `noexcept` doesn't actually prevent the function from throwing exceptions or calling other functions that are potentially throwing. Rather, when an exception is thrown in a `noexcept` function, `std::terminate` will be called.
- ❖ The `noexcept` specifier has an optional boolean parameter. `noexcept(true)` is equivalent to `noexcept` and `noexcept(false)` means the function is potentially throwing.

```
#include <iostream>

void func1(void) { throw 1; }

void func(void) noexcept { func1(); }

int main(void) {
    try { func(); }
    catch (...) { std::cout << "Not Called" << std::endl; }
    return 0;
}
```

Exception & Error Handling

- ❖ The **noexcept** as an operator can be used inside functions. It takes an expression as an argument, and returns true or false. The noexcept operator is evaluated at compile-time.
- ❖ The **noexcept operator** can be used to conditionally execute code depending on whether it is potentially throwing or not.
 - Use the noexcept specifier in cases where you want to express a no-fail guarantee.
 - If you are uncertain whether a function should have a no-fail/no-throw guarantee, do not mark it with noexcept.
- ❖ Dynamic resources shall be released properly when you use try-catch to handle exceptions

```
struct S {};  
void g(void){};  
void f(void) { throw -1; }  
void h(void) noexcept {};  
  
constexpr bool b1{noexcept(5 + 3)}; // true; ints are non-throwing  
constexpr bool b2{noexcept(f())}; // false; f() throws an exception  
constexpr bool b3{noexcept(g())}; // false; g() is implicitly noexcept(false)  
constexpr bool b4{noexcept(h())}; // true; h() is explicitly noexcept(true)  
constexpr bool b5{noexcept(S{})}; // true; a struct's default constructor is noexcept by default
```

```
try { auto *ptr{new Data{"int", 18}}; processData(ptr); delete ptr; }  
catch (...) {  
    std::cout << "Failed to process data" << std::endl;  
    /* If processData throws an exception data is not deleted. */  
}
```

Exception & Error Handling

- ❖ Destructors by default are `noexcept` functions and shall not throw exceptions. Throwing an exception from a destructor may call `std::terminate`.
 - Destruction during stack unwinding terminated by throwing an exception, calls `std::terminate()`.
- ❖ Exceptions decrease source code size and make source codes more readable
- ❖ Exceptions increase size and decrease performance of your executable
 - The main performance penalty for exceptions happens when an exception is actually thrown
- ❖ Exceptions are not safe and deterministic (exception objects are created dynamically)
 - Exception objects shall be accessible where they are created and also where they are caught
- ❖ Exceptions flow is unclear and complex
- ❖ Exceptions are not used in programming of safety-critical and deterministic systems
- ❖ To disable exception handling you can use **`-fno-exceptions`** when you compile your code.

C++ Language

❖ Some useful links

- [C++ Reference](#)
- [Standard C++ Library Reference](#)
- [C++ Tutorial](#)
- [C++ Language](#)
- [C++ Tutorial](#)
- [C++ Full Course For Beginners \(Learn C++ in 10 hours\)](#)
- [C++ Tutorial for Beginners - Full Course](#)
- [C++ Tutorial 2021](#)
- [C++ Programming Tutorials Playlist](#)